

Operating - System Structures

Practice Exercises

2.1 What is the purpose of system calls?

Answer:

System calls allow user-level processes to request services of the operating system.

2.2 What is the purpose of the command interpreter? Why is it usually separate from the kernel?

Answer:

It reads commands from the user or from a file of commands and executes them, usually by turning them into one or more system calls. It is usually not part of the kernel because the command interpreter is subject to changes.

2.3 What system calls have to be executed by a command interpreter or shell in order to start a new process on a UNIX system?

Answer: A `fork()` system call and an `exec()` system call need to be performed to start a new process. The `fork()` call clones the currently executing process, while the `exec()` call overlays a new process based on a different executable over the calling process.

2.4 What is the purpose of system programs?

Answer:

System programs can be thought of as bundles of useful system calls. They provide basic functionality to users so that users do not need to write their own programs to solve common problems.

2.5 What is the main advantage of the layered approach to system design? What are the disadvantages of the layered approach?

Answer:

As in all cases of modular design, designing an operating system in a

It is usually separate from the kernel because the command interpreter is subject to frequent changes and updates.

Advantages:

Modularity

Easy debugging :

Easy update :

No direct access to hardware :

Abstraction :

Disadvantage

Complex and careful implementation :

Slower in execution :

modular way has several advantages. The system is easier to debug and modify because changes affect only limited sections of the system rather than touching all sections. Information is kept only where it is needed and is accessible only within a defined and restricted area, so any bugs affecting that data must be limited to a specific module or layer. The primary disadvantage to the layered approach is the poor performance due to the overhead of traversing through the different layers to obtain a service provided by the operating system.

- 2.6** List five services provided by an operating system, and explain how each creates convenience for users. In which cases would it be impossible for user-level programs to provide these services? Explain your answer.

Answer:

The five services are:

- a. **Program execution.** The operating system loads the contents (or sections) of a file into memory and begins its execution. A user-level program could not be trusted to properly allocate CPU time.
- b. **I/O operations.** It is necessary to communicate with disks, tapes, and other devices at a very low level. The user needs only specify the device and the operation to perform on it, and the system converts that request into device- or controller-specific commands. User-level programs cannot be trusted to access only devices they should have access to and to access them only when they are otherwise unused.
- c. **File-system manipulation.** There are many details in file creation, deletion, allocation, and naming that users should not have to perform. Blocks of disk space are used by files and must be tracked. Deleting a file requires removing the name file information and freeing the allocated blocks. Protections must also be checked to assure proper file access. User programs could neither ensure adherence to protection methods nor be trusted to allocate only free blocks and deallocate blocks on file deletion.
- d. **Communications.** Message passing between systems requires messages to be turned into packets of information, sent to the network controller, transmitted across a communications medium, and reassembled by the destination system. Packet ordering and data correction must take place. Again, user programs might not coordinate access to the network device, or they might receive packets destined for other processes.
- e. **Error detection.** Error detection occurs at both the hardware and software levels. At the hardware level, all data transfers must be inspected to ensure that data have not been corrupted in transit. All data on media must be checked to be sure they have not changed since they were written to the media. At the software level, media must be checked for data consistency— for instance, whether the number of allocated and unallocated blocks of storage matches the total number on the device. There, errors are frequently processed independently (for instance, the corruption of data on a disk), so there must be a global program (the operating system) that handles all types of errors. Also, when errors are processed by the operating system, processes need not contain code to catch and correct all the errors possible on a system.

- 2.7 Why do some systems store the operating system in firmware, while others store it on disk?

Answer:

For certain devices, such as embedded systems, a disk with a file system may not be available for the device. In this situation, the operating system must be stored in firmware.

- 2.8 How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?

Answer:

Consider a system that would like to run both Windows and three different distributions of Linux (for example, RedHat, Debian, and Ubuntu). Each operating system will be stored on the disk. During system boot, a special program (which we will call the **boot manager**) will determine which operating system to boot into. This means that rather than initially booting to an operating system, the boot manager will first run during system startup. It is this boot manager that is responsible for determining which system to boot into. Typically, boot managers must be stored at certain locations on the hard disk to be recognized during system startup. Boot managers often provide the user with a selection of systems to boot into; boot managers are also typically designed to boot into a default operating system if no choice is selected by the user.

10E Exercises

- 2.9 The services and functions provided by an operating system can be divided into two main categories. Briefly describe the two categories, and discuss how they differ.

Answer:

One class of services provided by an operating system is to enforce protection between different processes running concurrently in the system. Processes are allowed to access only those memory locations that are associated with their address spaces. Also, processes are not allowed to corrupt files associated with other users. A process is also not allowed to access devices directly without operating system intervention. The second class of services provided by an operating system is to provide new functionality that is not supported directly by the underlying hardware. Virtual memory and file systems are two such examples of new services provided by an operating system.

- 2.10 Describe three general methods for passing parameters to the operating system.

Answer:

- Pass parameters in registers
- Registers pass starting addresses of blocks of parameters
- Parameters can be placed, or pushed, onto the stack by the program, and popped off the stack by the operating system.

- 2.11 Describe how you could obtain a statistical profile of the amount of

time a program spends executing different sections of its code. Discuss the importance of obtaining such a statistical profile.

Answer:

One could issue periodic timer interrupts and monitor what instructions or what sections of code are currently executing when the interrupts are delivered. A statistical profile of which pieces of code were active should be consistent with the time spent by the program in different sections of its code. Once such a statistical profile has been obtained, the programmer could optimize those sections of code that are consuming more CPU resources.

2.12 What are the advantages and disadvantages of using the same system-call interface for manipulating both files and devices?

Answer:

Each device can be accessed as though it was a file in the file system. Since most of the kernel deals with devices through this file interface, it is relatively easy to add a new device driver by implementing the hardware-specific code to support this abstract file interface. Therefore, this benefits the development of both user program code, which can be written to access devices and files in the same manner, and device-driver code, which can be written to support a well-defined API. The disadvantage of using the same interface is that it might be difficult to capture the functionality of certain devices within the context of the file access API, thereby resulting in either a loss of functionality or a loss of performance. Some of this could be overcome by the use of the `ioctl()` operation that provides a general-purpose interface for processes to invoke operations on devices.

2.13 Would it be possible for the user to develop a new command interpreter using the system-call interface provided by the operating system?

Answer:

A user should be able to develop a new command interpreter using the system-call interface provided by the operating system. The command interpreter allows a user to create and manage processes and also determine ways by which they communicate (such as through pipes and files). As all of this functionality could be accessed by a user-level program using the system calls, it should be possible for the user to develop a new command-line interpreter.

2.14 Describe why Android uses ahead-of-time (AOT) rather than just-in-time (JIT) compilation.

Answer:

Android uses ahead-of-time (AOT) compilation, a feature of the Android RunTime (ART) virtual machine environment in which Java applications are compiled to native machine code when they are installed on a system, rather than just-in-time when they are executed.

Java programs are first compiled to a Java bytecode `.class` file and then translated into an executable `.dex` file. Whereas many Java virtual machines perform just-in-time (JIT) compilation to improve application efficiency, ART performs ahead-of-time (AOT) compilation. Here, `.dex` files are compiled into native machine code when they are installed on a device, from which they can execute on the ART. AOT compilation allows more efficient application execution as well as reduced power consumption, features that are crucial for mobile systems.

2.15 What are the two models of interprocess communication? What are the

Advantages:

- Provides a consistent and familiar interface for users and developers to access both files and devices.
- Simplifies the addition of new device drivers by implementing hardware-specific code to support the abstract file interface.
- Allows user programs and device-driver code to be written to support a well-defined API.

Disadvantages:

- Some device functionality may be difficult to capture within the context of the file access API, resulting in a loss of functionality or performance.
- May require the use of a general-purpose interface like `ioctl()` to invoke operations on certain devices.

strengths and weaknesses of the two approaches?

Answer:

The two models of inter-process communication are the **message-passing model and the shared-memory model**. Message passing is useful for exchanging smaller amounts of data because no conflicts need to be avoided. **It is also easier to implement than shared memory** for inter-computer communication. **Shared memory allows maximum speed** and convenience of communication. Since it can be done at memory transfer speeds when it takes place within a computer. However, this method compromises protection and synchronization between the processes sharing memory.

2.16 Contrast and compare an application programming interface (API) and an application binary interface (ABI).

Answer:

The application programming interface (API) specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect. APIs specify certain functions at the application level.

At the architecture level, an application binary interface (ABI) is used to define how different components of binary code can interface for a given operating system on a given architecture. An ABI specifies low-level details, including address width, methods of passing parameters to system calls, the organization of the run-time stack, the binary format of system libraries, and the size of data types, just to name a few. Typically, an ABI is specified for a given architecture (for example, there is an ABI for the ARMv8 processor). Thus, an ABI is the architecture-level equivalent of an API. If a binary executable file has been compiled and linked according to a particular ABI, it should be able to run on different systems that support that ABI. However, because a particular ABI is defined for a certain operating system running on a given architecture, ABIs do little to provide cross-platform compatibility.

2.17 Why is the separation of mechanism and policy desirable?

Answer:

Mechanism and policy must be separate to ensure that systems are easy to modify. No two system installations are the same, so each installation may want to tune the operating system to suit its needs. With mechanism and policy separate, the policy may be changed at will while the mechanism stays unchanged. This arrangement provides a more flexible system.

2.18 **It is sometimes difficult to achieve a layered approach if two components of the operating system are dependent on each other. Identify a scenario in which it is unclear how to layer two system components that require tight coupling of their functionalities.**

Answer:

The virtual memory subsystem and the storage subsystem are typically tightly coupled and require a careful design in a layered system due to the following interactions. Many systems allow files to be mapped into the virtual memory space of an executing process. On the other hand, the virtual memory subsystem typically uses the storage system to provide the backing store for pages that do not currently reside in

API: An API is a set of rules, protocols, and tools that specifies how software components should interact with each other.

Focuses on specifying functions and their parameters at the application level.
Provides a high-level interface for software development.
Specifies functions available to application programmers.
Defines the expected behavior and usage of functions.
Helps in creating portable and reusable software components.
Provides flexibility to change the underlying implementation without affecting the application code.

ABI: An ABI, on the other hand, defines the low-level interface between different components of binary code.

Defines how different components of binary code interface at the architecture level.
Specifies low-level details of the interface between different components.
Includes details such as address width, parameter passing methods, run-time stack organization, binary format of system libraries, and data type sizes.
Ensures compatibility between binary code and the operating system on a specific architecture.
Facilitates binary compatibility, allowing executables compiled for a specific ABI to run on compatible systems.
Limited in providing cross-platform compatibility as it is specific to an operating system and architecture combination.

Separating mechanism and policy is desirable because it allows for easy modification and customization of systems. By keeping mechanism (how something is done) separate from policy (what is done), system installations can tune the operating system to their specific needs. This separation provides flexibility, allowing policy changes without affecting the underlying mechanism.

memory. Also, updates to the file system are sometimes buffered in physical memory before it is flushed to disk, thereby requiring careful coordination of the usage of memory between the virtual memory subsystem and the file system.

- 2.19 What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

Answer:

Benefits typically include the following: (a) adding a new service does not require modifying the kernel, (b) it is more secure as more operations are done in user mode than in kernel mode, and (c) a simpler kernel design and functionality typically results in a more reliable operating system. User programs and system services interact in a micro-kernel architecture by using inter-process communication mechanisms such as messaging. These messages are conveyed by the operating system. The primary disadvantages of the microkernel architecture are the overheads associated with inter-process communication and the frequent use of the operating system's messaging functions in order to enable the user process and the system service to interact with each other.

- 2.20 What are the advantages of using loadable kernel modules?

A loadable kernel module is a piece of code that can be dynamically loaded into the kernel at runtime to extend its functionality.

Answer:

It is difficult to predict what features an operating system will need when it is being designed. The advantage of using loadable kernel modules is that functionality can be added to and removed from the kernel while it is running. There is no need to either recompile or reboot the kernel.

Advantages of using loadable kernel modules include:

1. Dynamic extension and modification of kernel functionality without rebooting.
2. Efficient resource utilization and reduced memory footprint.
3. Simplified maintenance, customization, and debugging capabilities.

- 2.21 How are iOS and Android similar? How are they different?

Answer:

Similarities

- Both are based on existing kernels (Linux and Mac OSX).
- Both have an architecture that uses software stacks.
- Both provide frameworks for developers.

Differences

- iOS is closed-source, and Android is open-source.
- iOS applications are developed in Objective-C, and Android in Java.
- Android uses a virtual machine, and iOS executes code natively.

- 2.22 Explain why Java programs running on Android systems do not use the standard Java API and virtual machine.

Answer:

It is because the standard API and virtual machine are designed for desktop and server systems, not mobile devices. Google developed a separate API and virtual machine for mobile devices.

- 2.23 The experimental Synthesis operating system has an assembler incorporated in the kernel. To optimize system-call performance, the kernel assembles routines within kernel space to minimize the path that the system call must take through the kernel. This approach is the antithesis of the layered approach, in which the path through the kernel is extended to make building the operating system easier. Discuss the

pros and cons of the Synthesis approach to kernel design and system performance optimization.

Answer:

Synthesis is impressive due to the performance it achieves through on-the-fly compilation. Unfortunately, it is difficult to debug problems within the kernel due to the fluidity of the code. Also, such compilation is system-specific, making Synthesis difficult to port (a new compiler must be written for each architecture).

Advantages

Improved System Call Performance
Efficient Resource Utilization:

Disadvantages

Difficult Debugging:
Lack of Portability:

9E Exercises

- 2.2 What are the five major activities of an operating system with regard to process management?

Answer:

The five major activities are:

- The creation and deletion of both user and system processes
- The suspension and resumption of processes
- The provision of mechanisms for process synchronization
- The provision of mechanisms for process communication
- The provision of mechanisms for deadlock handling

- 2.3 What are the three major activities of an operating system with regard to memory management?

Answer:

The three major activities are:

- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- Allocate and deallocate memory space as needed.

- 2.4 What are the three major activities of an operating system with regard to secondary-storage management?

Answer:

The three major activities are:

- Free-space management.
- Storage allocation.
- Disk scheduling.

- 2.15 What are the five major activities of an operating system in regard to file management?

Answer:

The five major activities are:

- The creation and deletion of files
- The creation and deletion of directories
- The support of primitives for manipulating files and directories
- The mapping of files onto secondary storage
- The backup of files on stable (nonvolatile) storage media

7E Exercises

- 2.10** Why does Java provide the ability to call from a Java program native methods that are written in, say, C or C++? Provide an example of a situation in which a native method is useful.

Answer:

Java programs are intended to be platform I/O independent. Therefore, the language does not provide access to most specific system resources such as reading from I/O devices or ports. To perform a system I/O specific operation, you must write it in a language that supports such features (such as C or C++.) Keep in mind that a Java program that calls a native method written in another language will no longer be architecture-neutral.

- 2.13** In what ways is the modular kernel approach similar to the layered approach? In what ways does it differ from the layered approach?

Answer:

The modular kernel approach requires subsystems to interact with each other through carefully constructed interfaces that are typically narrow (in terms of the functionality that is exposed to external modules). The layered kernel approach is similar in that respect. However, the layered kernel imposes a strict ordering of subsystems such that subsystems at the lower layers are not allowed to invoke operations corresponding to the upper-layer subsystems. There are no such restrictions in the modular-kernel approach, wherein modules are free to invoke each other without any constraints.

- 2.14** What is the main advantage for an operating-system designer of using a virtual-machine architecture? What is the main advantage for a user?

Answer:

The system is easy to debug, and security problems are easy to solve. Virtual machines also provide a good platform for operating system research since many different operating systems may run on one physical system.

- 2.15** Why is a just-in-time compiler useful for executing Java programs?

Answer:

Java is an interpreted language. This means that the JVM interprets the bytecode instructions one at a time. Typically, most interpreted environments are slower than running native binaries, for the interpretation process requires converting each instruction into native machine code. A just-in-time (JIT) compiler compiles the bytecode for a method into native machine code the first time the method is encountered. This means that the Java program is essentially running as a native application (of course, the conversion process of the JIT takes time as well but not as much as bytecode interpretation.) Furthermore,

the JIT caches compiled code so that it may be reused the next time the method is encountered. A Java program that is run by a JIT rather than a traditional interpreter typically runs much faster.

- 2.16** What is the relationship between a guest operating system and a host operating system in a system like VMware? What factors need to be considered in choosing the host operating system?

Answer:

A guest operating system provides its services by mapping them onto the functionality provided by the host operating system. A key issue that needs to be considered in choosing the host operating system is whether it is sufficiently general in terms of its system-call interface in order to be able to support the functionality associated with the guest operating system.

Solution by D. M. Nadim Hayder